KRIES: Kernel Runtime Integrity Enforcement System

Your Name

Affiliation

KRIES: Kernel Runtime Integrity Enforcement System

Abstract

KRIES (Kernel Runtime Integrity Enforcement System) is a loadable Linux kernel module (LKM) designed to monitor, detect, and report anomalous runtime behavior from within the kernel itself. The system targets three primary threat surfaces: unauthorized process debugging via `ptrace`, suspicious kernel module activity, and abnormal process lifecycle events. KRIES exposes its findings to user space through the `/proc` virtual filesystem, enabling lightweight integration with existing monitoring pipelines. This paper describes the design rationale, implementation strategy, kernel APIs used, and the security trade-offs involved in building a host-based integrity monitor at the kernel level.

Introduction

Modern operating systems face a persistent challenge: software running in user space cannot fully trust the environment it operates in. Rootkits, memory-resident malware, and debugging-based exploits operate below the visibility horizon of user-space security tools. Traditional antivirus and endpoint detection software suffers from a fundamental limitation — it is itself a user-space process, subject to manipulation by the very threats it tries to detect.

Kernel-level monitoring addresses this gap. By operating at ring 0 (the highest CPU privilege level), a kernel module can observe system state that is invisible or falsifiable from user space. It can walk the real process table, inspect `task_struct` flags directly, enumerate loaded modules from the kernel's own linked list, and react to events before they are mediated by any user-space layer.

KRIES is built on this principle. It is not a replacement for a full security framework (such as SELinux or AppArmor), but rather an instructional and functional demonstration of

how kernel-level runtime integrity monitoring works. The project is structured in progressive phases, each introducing a new monitoring capability while keeping the code readable and auditable.

Goals

- Demonstrate the structure and lifecycle of a Linux loadable kernel module.
- Implement non-invasive monitoring of process and module state.
- Detect common indicators of compromise (ptrace-based debugging, unexpected modules).
- Expose system state to user space via the `/proc` interface.
- Serve as a reference implementation for kernel security concepts.

Non-Goals

- KRIES does not enforce policy (it does not block processes or kill threads).
- KRIES does not use kernel function hooking (kprobes, ftrace trampolines) in its base implementation.
- KRIES is not intended for production deployment without further hardening.

Background & Prior Work

Loadable Kernel Modules (LKMs)

Linux supports dynamic extension of the kernel through loadable modules. A module is an ELF object file (`.ko`) that is linked into the running kernel image at load time. Modules share the kernel's address space and execute at ring 0. They are used to implement device drivers, filesystems, network protocols, and security subsystems.

The module lifecycle is:

```
Compilation → insmod (load) → [running in kernel] → rmmod (unload)
```

Modules are managed by `module_init()` and `module_exit()` callbacks, registered at compile time via macros. The kernel verifies module compatibility via a version magic string embedded in the `.ko` file.

The Linux Process Model

Every process in Linux is represented by a `task_struct` — a large kernel data structure containing all information about a running process: its PID, state, credentials, memory maps, file descriptors, and scheduling information. All `task_struct` instances are linked together in a doubly-linked circular list, iterable via the `for_each_process()` macro.

ptrace and Debugging Detection

`ptrace` is the Linux system call that underlies all user-space debuggers (GDB, strace, ltrace). When a process is being traced, its `task_struct` has the `PT_PTRACED` flag set in the `.ptrace` field. Inspecting this field from kernel space provides a reliable, tamper-resistant method of detecting active debugging.

From user space, a process can call `ptrace(PTRACE_TRACEME, ...)` to voluntarily allow itself to be traced, or a debugger can attach to a target via `ptrace(PTRACE_ATTACH, ...)`. In both cases, the kernel sets the flag on the target's `task_struct`.

Anti-debugging techniques frequently use ptrace as either a detection vector or an attack surface. Malware may call `ptrace(PTRACE_TRACEME)` on itself to prevent a second debugger from attaching, or it may attempt to detect and terminate itself if a debugger is found.

The /proc Virtual Filesystem

`/proc` is a pseudo-filesystem that exists only in memory. It is the standard Linux mechanism for exposing kernel state to user space. Each entry under `/proc` is backed by a kernel data structure with registered read (and optionally write) callbacks. When a user runs `cat /proc/kries`, the kernel calls the registered read function and streams the output to the user's terminal.

Related Systems

System	Approach	Scope
SELinux	Mandatory Access Control via LSM hooks	Policy enforcement
AppArmor	Profile-based MAC	Application confinement
Auditd	Syscall auditing via the audit subsystem	Event logging
LKRG (Linux Kernel Runtime Guard)	Integrity checking of kernel data structures	Kernel self-protection
Sysdig / Falco	eBPF-based syscall monitoring	Runtime threat detection
KRIES	LKM-based state inspection and detection	Monitoring and alerting

KRIES occupies a simpler, more transparent niche than production systems like LKRG or Falco. It is designed to be readable and instructional while still performing real detection work.

Threat Model

KRIES is designed to detect the following classes of threats:

1. Unauthorized Process Debugging

Actor: An attacker or malicious process using `ptrace` to inspect or manipulate another process's memory or execution.

Indicator: `task_struct.ptrace` field has `PT_PTRACED` set for a process that should not be under debugger control.

Risk: Code injection, credential theft, anti-tamper bypass, exploit development.

2. Suspicious Kernel Module Loading

Actor: A rootkit or privileged malware loading a kernel module to hide itself or intercept system calls.

Indicator: A module appears in the kernel module list with an unrecognized name, or a module is loaded from an unexpected path.

Risk: System call table hooking, process hiding, network traffic interception.

3. Abnormal Process State

Actor: A process in an unexpected lifecycle state (zombie, stopped) that persists longer than expected.

Indicator: Processes with unusual flags or parent-child relationships visible in `task_struct` iteration.

Risk: Process injection staging, resource exhaustion, covert persistence.

Out of Scope

- Network-based threats
- Filesystem integrity (handled better by IMA/EVM)
- Hardware-level attacks
- Kernel exploits that compromise KRIES itself

System Architecture

instert photo here

Component Responsibilities

Component	Responsibility
Logging Infrastructure	Unified `printk` wrappers with consistent formatting
Process Monitor	Iterates `task_struct` list; extracts PID, name, parent, flags
Module Monitor	Iterates kernel module linked list; logs names and states
Detection Engine	Applies rules to monitor output; generates structured alerts
/proc Interface	Exposes detection results and system state to user space

---

Module Design (Phase Breakdown)

Phase 1 — Minimal Kernel Module

Purpose. Establish the scaffolding for all subsequent phases. Verify that the build environment is correctly configured and that the module can be loaded and unloaded cleanly.

Key Kernel APIs.

API	Header	Description
`module_init(fn)`	`<linux/module.h>`	Register the init function
`module_exit(fn)`	`<linux/module.h>`	Register the exit function
`printk(level, fmt, ...)`	`<linux/kernel.h>`	Write to kernel ring buffer
`MODULE_LICENSE("GPL")`	`<linux/module.h>`	Declare license (required for GPL symbols)

Design Notes. The `__init` and `__exit` annotations on the init/exit functions are compiler hints. `__init` marks code that can be freed from memory after initialization completes. `__exit` marks code that is only needed during unload. Both annotations reduce the module's runtime memory footprint.

The `MODULE_LICENSE("GPL")` declaration is mandatory for any module that uses GPL-only exported kernel symbols. Without it, the kernel will refuse to resolve those symbols and `insmod` will fail with an "Unknown symbol" error. It also taints the kernel if omitted, which affects crash dump analysis.

Build System. KRIES uses the kernel's `kbuild` system. The `Makefile` delegates compilation to the kernel's own build infrastructure at `/lib/modules/$(uname -r)/build`. This ensures the module is compiled with the exact same flags, headers, and ABI as the running kernel — a requirement for successful loading.

```
obj-m += kries.o
KDIR  := /lib/modules/$(shell uname -r)/build
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
```

Phase 2 — Logging Infrastructure

Purpose. Replace raw `printk()` calls with structured, consistent logging macros. All KRIES output must be identifiable in `dmesg` output by a consistent prefix.

Design. Three log levels are defined, each mapping to a `printk` severity level:

Macro	printk Level	Use Case
`KRIES_INFO(fmt, ...)`	`KERN_INFO`	Normal operational messages
`KRIES_WARN(fmt, ...)`	`KERN_WARNING`	Suspicious but non-critical events

Macro	printk Level	Use Case
`KRIES_ALERT(fmt, ...)`	`KERN_ALERT`	High-severity detections

All macros prepend `[KRIES]` to the message and append `\n` automatically, ensuring consistent formatting regardless of the calling site.

Design Notes. Using macros rather than wrapper functions keeps logging zero-overhead in release builds (the compiler inlines them). The `##__VA_ARGS__` pattern handles the case where no variadic arguments are passed, avoiding a trailing-comma compile error in C99.

The `KERN_ALERT` level is intentionally distinct from `KERN_CRIT` and `KERN_EMERG` — those levels are reserved for kernel self-protection failures. `KERN_ALERT` is the appropriate level for "action must be taken immediately" security events that are serious but do not indicate kernel corruption.

Phase 3 — Process Monitoring

Purpose. Enumerate all running processes using kernel-internal data structures and log their identity and relationships.

Key Kernel APIs.

API	Header	Description
`for_each_process(task)`	`<linux/sched/signal.h>`	Iterate over all `task_struct` entries
`task->pid`	`<linux/sched.h>`	Process ID
`task->comm`	`<linux/sched.h>`	Process name (up to 16 chars)

API	Header	Description
`task->parent->pid`	`<linux/sched.h>`	Parent process ID
`rcu_read_lock()`	`<linux/rcupdate.h>`	Acquire RCU read lock
`rcu_read_unlock()`	`<linux/rcupdate.h>`	Release RCU read lock

Design Notes. RCU (Read-Copy-Update) is the kernel's primary mechanism for protecting shared data structures that are read far more often than they are written. The process list is protected by RCU. Any iteration over it must be wrapped in an RCU read-side critical section (`rcu_read_lock()` / `rcu_read_unlock()`). Failing to do so can result in use-after-free bugs if a process exits during iteration.

`task->comm` is a 16-byte character array (`TASK_COMM_LEN`). It stores a truncated version of the process's executable name. It is not the same as the full command line (which lives in user-space memory and requires `copy_from_user()` to access safely).

The `for_each_process()` macro only iterates over thread group leaders (one entry per process). To iterate over all threads within a process, `for_each_thread()` would be used instead.

Phase 4 — Debug Detection

Purpose. Identify processes that are currently under debugger control by inspecting `ptrace`-related flags in `task_struct`.

Detection Mechanism.

```
/* In task_struct, the .ptrace field holds bitflags. */
/* PT_PTRACED is set when a process has an active tracer attached. */

if (task->ptrace & PT_PTRACED) {
```

```
    /* This process is being traced (debugged). */
}
```

The `ptrace` field in `task_struct` is a bitmask. The relevant flags are:

Flag	Value	Meaning
`PT_PTRACED`	`0x00000001`	Process has an active tracer
`PT_SEIZED`	`0x00010000`	Tracer used `PTRACE_SEIZE` (newer attach method)
`PT_STOPPED`	`0x00000004`	Process is in ptrace-stop state

Design Notes. This detection method is reliable when KRIES runs at a higher privilege level than the traced process (which is always true for kernel code). It cannot be bypassed from user space by simply modifying the flag — `task_struct` lives in kernel memory.

However, a kernel rootkit running at the same privilege level could potentially clear these flags before KRIES reads them. This is a fundamental limitation of same-ring monitoring, addressed further in the Limitations section.

False Positives: Any process being debugged by a legitimate debugger (GDB, strace) will be flagged. In a real deployment, a whitelist of expected traced processes (or processes traced by authorized debuggers) would be necessary. KRIES logs all detected cases and lets the operator decide.

Phase 5 — Kernel Module Monitoring

Purpose. Enumerate all currently loaded kernel modules using the kernel's internal module list.

Key Kernel APIs.

API	Header	Description
`struct module`	`<linux/module.h>`	Represents a loaded kernel module
`THIS_MODULE`	`<linux/module.h>`	Pointer to the current module's `struct module`
`list_for_each_entry()`	`<linux/list.h>`	Iterate over a `list_head` linked list
`mutex_lock(&module_mutex)`	`<linux/mutex.h>`	Lock protecting the module list

Module States. The `struct module` contains a `state` field of type `enum module_state`:

State	Value	Meaning
`MODULE_STATE_LIVE`	0	Module is running normally
`MODULE_STATE_COMING`	1	Module is being loaded
`MODULE_STATE_GOING`	2	Module is being unloaded
`MODULE_STATE_UNFORMED`	3	Module is being initialized

Design Notes. The kernel's module list is a standard doubly-linked list (`list_head`) rooted at an internal symbol. Traversal requires holding `module_mutex` to prevent concurrent modifications (module load/unload from another context).

`struct module` contains the module's name in the `.name` field — a fixed-length character array. The module list includes KRIES itself, which should be expected and can be filtered from output.

A rootkit commonly attempts to remove its own `struct module` from this list to become invisible to `lsmod`. KRIES would then not see it either — which is why this monitoring technique is most useful for detecting accidental or semi-covert loads, not fully-committed rootkits. LKRG and similar tools use memory integrity checks to detect such list tampering.

Phase 6 — /proc Interface

Purpose. Create a readable file at `/proc/kries` that streams KRIES status information to any user-space process that opens it.

Key Kernel APIs.

API	Header	Description
`proc_create()`	`<linux/proc_fs.h>`	Create a /proc entry
`proc_remove()`	`<linux/proc_fs.h>`	Remove a /proc entry
`seq_file` interface	`<linux/seq_file.h>`	Safe sequential output for /proc files
`seq_printf()`	`<linux/seq_file.h>`	printf-like output to seq_file buffer
`single_open()`	`<linux/seq_file.h>`	Simplified seq_file open for single-pass output

Design Notes. Early Linux `/proc` files used a raw `read_proc` callback that required careful buffer management. The modern approach uses the `seq_file` interface, which handles buffer allocation, pagination, and re-reading automatically. It is the correct API for any `/proc` file that outputs more than a trivial amount of data.

The `/proc` file created by KRIES is **read-only** (`0444` permissions). Write support (for runtime configuration) could be added via a `write` callback in the `file_operations`

struct, but is deferred to future work.

Cleanup: The `/proc` entry must be explicitly removed in `kries_exit()` via `proc_remove()`. Failing to do so leaves a dangling pointer in the proc tree — accessing `/proc/kries` after the module unloads would cause a kernel panic (null dereference into freed module memory).

Phase 7 — Detection Engine

Purpose. Combine process and module monitoring into a rule-based detection system that generates structured alerts.

Rule Architecture. Each detection rule is a boolean function with a consistent signature:

```
bool is_debugged(struct task_struct *task);
bool is_suspicious_module(struct module *mod);
```

A central `kries_run_scan()` function iterates over all processes and modules, applying each rule and logging structured alerts when a rule fires.

Alert Format.

```
[KRIES][ALERT] type=DEBUG_DETECTED pid=<pid> name=<comm> parent=<ppid>
[KRIES][ALERT] type=SUSPICIOUS_MODULE name=<modname> state=<state>
```

Structured key=value log format is used rather than free-form text. This makes alerts parseable by log aggregation tools (Splunk, ELK, journald structured logging) without post-processing.

Design Notes. The detection engine is intentionally stateless in its initial implementation — each scan is an independent snapshot. A stateful engine (detecting changes between scans, rather than absolute state) would require persisting previous scan

results and comparing deltas. This is a natural extension for Phase 8+ work.

The scan is triggered on module load (and can be extended to run on a timer via `timer_list` or a kernel thread via `kthread_create()`). Running continuously in a kernel thread would require careful CPU usage management to avoid starving other kernel work.

Kernel Concepts Reference

This section provides brief explanations of key kernel concepts used throughout KRIES, for readers new to kernel development.

task_struct

The central data structure in the Linux process model. Every process and thread has exactly one `task_struct`. It contains:

- **Identity:** `pid`, `tgid`, `comm` (name), `cred` (credentials)
- **State:** `__state` (running, sleeping, stopped, zombie, etc.)
- **Relationships:** `parent`, `children` list, `sibling` list
- **Memory:** `mm` (memory descriptor, NULL for kernel threads)
- **Scheduling:** priority, CPU affinity, runtime statistics
- **Debugging:** `ptrace` flags

Ring Buffer (dmesg)

The kernel maintains a circular buffer of log messages written via `printk()`. This buffer is accessible from user space via `dmesg` or `/dev/kmsg`. When the buffer fills, the oldest messages are overwritten. The buffer is stored in kernel memory and survives across `insmod`/`rmmod` cycles but not across reboots.

RCU (Read-Copy-Update)

A synchronization mechanism that allows multiple readers to proceed concurrently without locking, while writers operate on a private copy and "publish" updates atomically.

The process list uses RCU. Readers must call `rcu_read_lock()` before iterating and `rcu_read_unlock()` when done.

GPL Symbol Export

Many kernel functions are exported only to GPL-licensed modules (`EXPORT_SYMBOL_GPL()`). This is a licensing mechanism — using these symbols in a non-GPL module is a license violation, and the kernel will refuse to load such a module. KRIES declares `MODULE_LICENSE("GPL")` to comply.

Kernel Taint

The kernel tracks "taint" flags that indicate its trustworthiness for debugging purposes. Loading a non-GPL module (`TAINT_PROPRIETARY_MODULE`), loading an unsigned module (`TAINT_UNSIGNED_MODULE`), or encountering a kernel oops sets taint flags visible in `/proc/sys/kernel/tainted`. Tainted kernels produce less useful crash reports.

Security Considerations

Privilege Requirements

Loading KRIES requires root access (`CAP_SYS_MODULE` capability). This is an appropriate gate — a module with read access to all `task_struct` data and the ability to write to `/proc` must not be loadable by unprivileged users.

Read-Only Design

KRIES does not modify any kernel data structures. It is a passive observer. This design choice:

- Minimizes the risk of kernel panics from incorrect pointer manipulation
- Reduces the attack surface KRIES itself presents
- Makes the module easier to audit

Secure Boot Compatibility

On systems with UEFI Secure Boot enabled, the kernel requires all modules to be signed with a trusted key. KRIES, as a development module, will not be signed by default. Loading it requires either:

1. Disabling Secure Boot in BIOS/UEFI firmware settings
2. Enrolling a custom Machine Owner Key (MOK) and signing `kries.ko` with it using `mokutil` and `sign-file`

For production deployment, option 2 is strongly preferred.

/proc Permissions

The `/proc/kries` entry is created with `0444` (world-readable) permissions in the base implementation. In a hardened deployment, this should be restricted to `0400` or `0440` (root-only or root+group) to prevent information disclosure about system state to unprivileged users.

Limitations

Same-Ring Monitoring

KRIES operates at ring 0 (kernel privilege level), the same level as any kernel rootkit it is trying to detect. A sufficiently capable rootkit can:

- Remove itself from the module list before KRIES scans it
- Clear `PT_PTRACED` flags before KRIES reads them
- Hook the functions KRIES calls to return falsified data

This is the fundamental limitation of in-kernel monitoring: the monitor and the threat share the same trust boundary. Solutions include:

- Hardware virtualization (running the monitor in a hypervisor layer below the OS)
- Remote attestation (measuring kernel state from an external trusted system)
- Kernel self-integrity mechanisms (LKRG, IMA)

Snapshot-Based Detection

KRIES performs point-in-time scans. A threat that exists between scans will be missed. Continuous monitoring would require a persistent kernel thread, which introduces CPU overhead and scheduling complexity.

No Enforcement

KRIES logs threats but does not block or terminate them. Adding enforcement (e.g., sending SIGKILL to a detected process) would require careful design to avoid killing legitimate processes and to handle race conditions between detection and termination.

task_struct Stability

`task_struct` is an internal kernel data structure. Its layout and field names can change between kernel versions. KRIES must be recompiled against the kernel headers for each target kernel version. Fields deprecated or renamed in newer kernels will cause compilation failures.

Future Work

Feature	Description	Kernel Mechanism
Continuous Monitoring	Run scans on a periodic timer or in a dedicated kernel thread	`kthread_create()`, `timer_list`
Syscall Monitoring	Intercept specific system calls	`kprobes`, LSM hooks
File Integrity	Monitor critical file modifications	`inotify` (user space) or `fsnotify` (kernel)

Feature	Description	Kernel Mechanism
Network Monitoring	Log suspicious outbound connections	Netfilter hooks
Alert Persistence	Store alerts in a ring buffer for `/proc` reads	`kfifo`
Runtime Configuration	Allow enabling/disabling rules via `/proc` writes	`write` file operation
Whitelist Support	Skip known-good processes/modules	Hash table of approved names
Kernel Thread Monitoring	Detect unexpected kernel threads	`for_each_thread()` + kthread flag inspection
Module Signing Verification	Warn on unsigned module loads	`module->sig_ok` field

---

Conclusion

KRIES demonstrates that meaningful runtime integrity monitoring is achievable within the kernel itself using stable, well-documented Linux kernel APIs. By operating at ring 0, KRIES observes system state that is inaccessible or falsifiable from user space, providing a stronger foundation for detecting low-level threats such as ptrace-based debugging and unauthorized kernel modules.

The phased development approach — from a minimal module skeleton through to a structured detection engine with a `/proc` interface — illustrates how kernel security tools are built incrementally, with each layer providing the foundation for the next.

While KRIES is not a production-grade security tool (lacking enforcement, continuous monitoring, and tamper resistance against sophisticated rootkits), it is a complete

and functional demonstration of host-based kernel integrity monitoring principles. It serves as both a learning platform and a starting point for more advanced kernel security research.

References

1. Corbet, J., Rubini, A., & Kroah-Hartman, G. (2005). *Linux Device Drivers, 3rd Edition*. O'Reilly Media. https://lwn.net/Kernel/LDD3/

2. Love, R. (2010). *Linux Kernel Development, 3rd Edition*. Addison-Wesley Professional.

3. The Linux Kernel documentation project. *The Linux Kernel API*. https://www.kernel.org/doc/html/latest/

4. The Linux Kernel documentation project. *Loadable Kernel Module HOWTO*. https://www.kernel.org/doc/html/latest/kbuild/modules.html

5. McKenney, P. E. (2007). *What is RCU, Fundamentally?*. LWN.net. https://lwn.net/Articles/262464/

6. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.

7. Linux Kernel source — `include/linux/sched.h`, `include/linux/module.h`, `include/linux/proc_fs.h`. https://elixir.bootlin.com/linux/latest/source

8. LKRG (Linux Kernel Runtime Guard) project. https://lkrg.org/

9. Falco runtime security project. https://falco.org/

10. National Institute of Standards and Technology. (2022). *Guide to Enterprise Patch Management Planning* (SP 800-40r4). NIST.

End of Document